

SafeExec Midpoint Technical Evidence Review

AI-use disclosure. Drafted with Codex (GPT-5, Codex desktop app) from student-authored project artifacts and command outputs generated in this repository. AI-drafted, student-revised. Full audit trail: `docs/ai-use-log.md`.

Course: CISC 699 Applied Project in Computer Information Sciences

Student: Zixuan Liang

Advisor: Prof. Khalid Lateef

Instructor: Dr. Majid Shaalan

Assignment: 08 Hard Stop 4: Midpoint Technical Evidence Review

Prepared: 2026-06-19

1. Technical Question

The most important midpoint evidence question is:

Can the current SafeExec execution path run a controlled batch of normal, failure-control, and containment-oriented Python programs repeatably across the development local backend, hardened Docker backend, and gVisor backend, and what does the timing evidence say about the project's success criteria for the second half of the course?

This question matters because the approved project claim is not simply that a `/execute` API exists. The claim is that SafeExec can measure sandbox behavior under meaningful conditions and eventually compare hardened Docker against gVisor. The midpoint package therefore emphasizes repeatable evidence, backend comparison, and honest interpretation of the first visible weakness.

2. Evidence Collection Point

The evidence was collected at the execution-service boundary: the same `ExecutionRequest` model was passed through `safeexec.service.execute_code()`, which selected the requested backend and returned a structured `ExecutionResult`.

```

Benchmark script
-> ExecutionRequest(code, backend, limits)
-> safeexec.service.execute_code()
-> LocalSubprocessBackend | DockerBackend | DockerBackend(runtime=runcsc)
-> ExecutionResult(status, exit_code, stdout, stderr, duration_ms,
containment_reason)

```

This maps directly to the W4 design review architecture: the test harness is outside the sandbox; the service layer is the stable API boundary; the backend is the subsystem under review; the result schema is the evidence surface used for later functional, adversarial, and performance reporting.

3. Method

3.1 Script and repeatability

The benchmark harness is `scripts/run_midpoint_evidence.py`. It emits:

- `midpoint-results.json` with raw per-run records, expectations, result objects, and pass/fail classifications.
- `midpoint-results.csv` with one row per execution.
- `midpoint-summary.csv` and `midpoint-summary.md` with grouped pass rates and timing summaries.
- Environment snapshots for local and target-host execution.

The script was run with **5 repeats**. Each repeat executed applicable cases for the requested backend. Container-only containment probes were intentionally not run on the local subprocess backend because the local backend is a development shim, not a security boundary.

3.2 Case set

The W8 batch includes 10 cases:

- **Correctness:** `functional_hello`, `deterministic_arithmetic`, `json_serialization`, and `stderr_exit_code`.
- **Failure control:** `wall_timeout` and `output_truncation`.
- **Containment:** `container_nonroot_uid`, `container_readonly_root`, `tmpfs_write_allowed`, and `network_disabled_probe`.

The correctness cases verify normal output, deterministic computation, standard library import/use, and non-zero exit reporting. The failure-control cases

verify timeout reporting and output-limit truncation. The containment cases verify non-root execution, read-only root behavior, writable tmpfs behavior, and disabled network egress.

3.3 Environments

Environment	Role	Evidence files
macOS authoring machine	Local development backend, unit tests, document generation	evidence-local/environment-snapshot.txt, evidence-local/local-test-output.txt, evidence-local/midpoint-summary.md
Ubuntu target host	Docker and gVisor backend comparison	evidence-target/environment-snapshot.txt, evidence-target/midpoint-summary.md, evidence-target/docker-image-python311.txt, evidence-target/runsc-version.txt

Target-host toolchain facts:

- Linux kernel: 5.15.0-179-generic x86_64
- Docker: Docker version 29.5.0
- gVisor: runsc version release-20260511.0
- Container image: python:3.11-slim, image ID ae52c5bef62a
- Target repository baseline: 674fa58f5a21b35a7121fdcf990725a761cce23
- Benchmark script hash: recorded in

evidence-target/run-midpoint-evidence-sha256.txt

The target host still lacks make, and the host Python is 3.10.12. The container backends execute Python 3.11 inside python:3.11-slim, so the W8 container evidence remains aligned with the approved project target even though the host-side harness should be moved to Python 3.11 or a dev container before the final benchmark.

4. Results

4.1 Backend-level summary

Backend	Passed / Total	Pass rate	Mean duration	Median duration	Maximum duration
Local subprocess	30 / 30	100.0%	93.2 ms	32.9 ms	402.0 ms

Hardened Docker	50 / 50	100.0%	868.8 ms	771.7 ms	1985.4 ms
gVisor (runsc)	49 / 50	98.0%	1361.8 ms	1327.5 ms	3167.2 ms

The local backend is much faster but is not containment evidence. Docker passed all current W8 cases. gVisor passed all correctness, timeout, output truncation, non-root, read-only-root, and network-disabled cases, but one `tmpfs_write_allowed` run timed out at the current wall-time boundary.

4.2 Target-host category summary

Backend	Category	Passed / Total	Mean duration	Median duration	Max duration
Docker	Containment	20 / 20	991.6 ms	864.2 ms	1985.4 ms
Docker	Correctness	20 / 20	635.5 ms	618.0 ms	1157.2 ms
Docker	Failure control	10 / 10	1090.0 ms	1220.1 ms	1403.8 ms
gVisor	Containment	19 / 20	1612.4 ms	1505.2 ms	3167.2 ms
gVisor	Correctness	20 / 20	1114.5 ms	1083.6 ms	2352.5 ms
gVisor	Failure control	10 / 10	1355.2 ms	1403.4 ms	1777.9 ms

The full case-level table is intentionally submitted as raw supporting evidence instead of being compressed into the prose document. See `evidence-target/midpoint-summary.md` and `evidence-target/midpoint-results.csv` for the per-case rows and exact durations.

5. Interpretation

5.1 Positive signals

The current project is now beyond demonstration. It can produce structured, repeatable evidence across the three execution modes used in the design review: local development, hardened Docker, and gVisor. The raw results show that the core service API and result schema are stable enough to support benchmark and adversarial expansion.

Docker is the strongest W8 signal. It passed all 50 target-host records across correctness, failure-control, and containment probes. The current hardened

command choices are being exercised in real execution: network disabled, non-root user, read-only root filesystem, tmpfs-only write path, output truncation, and wall-time enforcement.

gVisor is also technically alive. It passed 49 of 50 records and all of the most security-relevant probes in this small set: non-root execution, read-only-root rejection, network-disabled behavior, timeout reporting, and output truncation.

5.2 Weaknesses and limitations

The main weakness is gVisor timing variability under the current timeout budget.

One tmpfs_write_allowed gVisor run timed out, and the gVisor containment cases had visibly higher tail latency than Docker. This does not prove that gVisor is functionally wrong; it shows that the current wall-time limit is too tight for a backend with higher startup overhead. For the next sprint, the benchmark should separate **program time budget** from **container startup allowance** or report cold-start and warm-run conditions separately.

The evidence set is still small relative to the final success criteria. It is a controlled midpoint batch, not the final HumanEval/MBPP functional subset and not the final adversarial suite. The current results support the claim that the measurement path works, but they do not yet support final claims such as ">=99% functional pass rate" or ">=95% adversarial containment under gVisor."

The validation host remains operationally imperfect. It has Docker and gVisor, but host Python is 3.10.12 and make is missing. This does not invalidate the container evidence, but it does mean the final reproducibility path needs to be tightened before W10-W14.

5.3 Midpoint adjustment

The second half of the course should prioritize evidence quality over new features. Based on this run, the next work should be:

1. Add a benchmark mode that records cold-start vs warm-start conditions explicitly.
2. Install or containerize the host-side benchmark harness so it runs under

Python 3.11 with consistent commands.

3. Expand the case set from 10 controlled programs to the planned functional corpus subset and adversarial taxonomy categories.

4. Add per-run metadata for Docker image digest, runtime, case category, timeout budget, and whether image pull/warmup was excluded from timing.

5. Treat gVisor tail latency as an engineering risk and adjust timeout interpretation before claiming performance success.

6. Link to Success Criteria

Success criterion from approved scope	W8 evidence status	Interpretation
Runnable service/API path	Supported	Service and backend path produced structured results repeatedly.
Hardened Docker containment baseline	Early positive signal	Docker passed all current W8 containment probes, but suite size is still small.
gVisor fallback/stronger isolation path	Partially supported	gVisor executes and contains probes, but one timing failure requires benchmark-method adjustment.
Functional correctness target	Not yet final	Correctness cases passed, but HumanEval/MBPP subset has not yet been integrated.
Adversarial containment target	Not yet final	Current probes are adversarial seeds, not the full taxonomy.
Reproducibility and evidence quality	Improving	Raw JSON/CSV, environment snapshots, script hash, and target metadata are included. Host tooling still needs cleanup.

7. Evidence Files Submitted

Primary package files:

- Midpoint-Technical-Evidence-Review.docx
- Midpoint-Technical-Evidence-Review.pdf
- midpoint-technical-evidence-review.md
- known-issues-risk-log.md
- canvas-submission-checklist.md

Supporting evidence:

- evidence-local/environment-snapshot.txt
- evidence-local/local-test-output.txt
- evidence-local/midpoint-results.json
- evidence-local/midpoint-results.csv

- evidence-local/midpoint-summary.md
- evidence-target/environment-snapshot.txt
- evidence-target/midpoint-results.json
- evidence-target/midpoint-results.csv
- evidence-target/midpoint-summary.md
- evidence-target/docker-image-python311.txt
- evidence-target/runsc-version.txt
- evidence-target/run-midpoint-evidence-sha256.txt

8. Conclusion

SafeExec is on track technically, but the midpoint evidence also clarified what must improve. The Docker backend is stable enough for broader functional and adversarial testing. The gVisor backend is integrated and mostly passing, but it needs better timing methodology before performance conclusions are defensible. The next sprint should therefore expand coverage while also improving measurement discipline: repeat counts, corpus provenance, startup/warm-run separation, and explicit limitation reporting.